

CVIS Hands-on Training on Development of Machine Learning and Deep Learning Models Summer 2025

Facilitator

Md. Fahmid-Ul-Alam Juboraj

Research Assistant



Inspiring Excellence



Inspiring Innovation

Computer Vision and Intelligent Systems Lab
Dept. Of Computer Science and Engineering
School of Data and Sciences, BRAC University

Deep Learning for Video Processing Part-2



What is Video Processing?

- Video processing deals with preparing and analyzing sequences of frames (images) to extract meaningful information.
- It involves operations such as resizing, filtering, or converting formats to make videos suitable for further tasks.
- Think of it as the step between raw camera footage and AI-ready data.



Why Do We Need Video Processing?

- Raw videos are large, noisy, and inconsistent (different formats, resolutions, frame rates).
- Preprocessing ensures videos are standardized so that models can learn effectively.
- It reduces computational cost and helps improve the accuracy and reliability of AI systems.



Applications of Video Processing in AI

- Security & Surveillance: detecting suspicious activities or anomalies in CCTV feeds.
- Healthcare: analyzing medical videos such as endoscopy or surgery recordings.
- Entertainment: automatic highlight generation for sports or movie summarization.
- Human-Computer Interaction: gesture recognition in AR/VR or sign language translation.

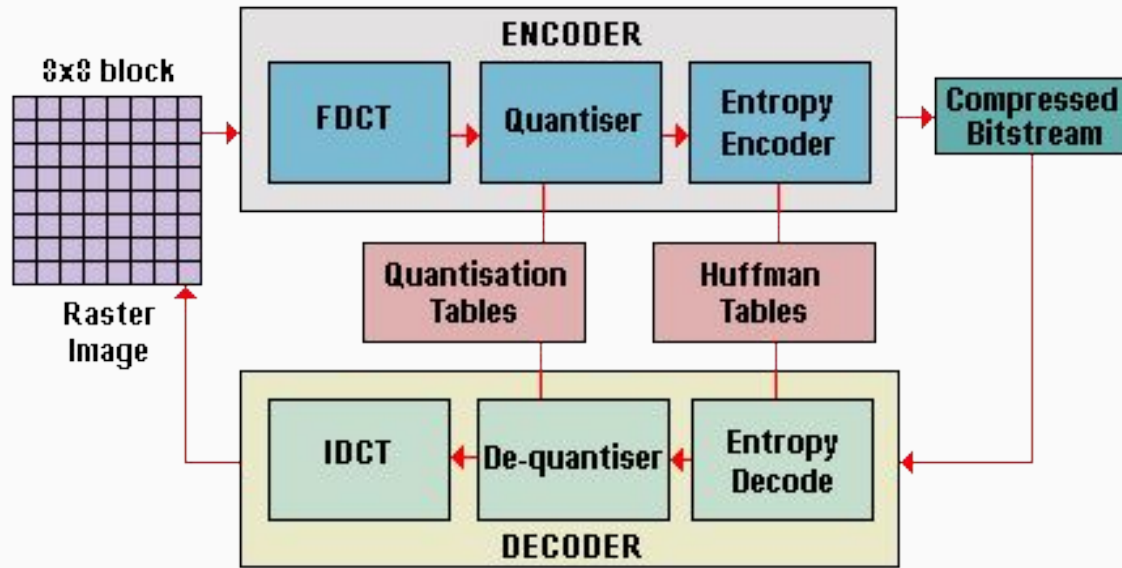


Video Formats & Conversion

- Raw videos come in different container formats (MP4, AVI, MOV, MKV), each supporting different codecs.
- Conversion ensures all videos are stored in standardized codec + container (e.g., MP4 + H.264/H.265) before preprocessing.
- Tools: FFmpeg for conversion, OpenCV for reading/writing.
- Example: Converting .avi (uncompressed, huge size) → .mp4 (H.264, efficient, GPU-friendly)

Video Formats & Conversion

Format	Typical Codec	Compression Type	Notes for DL
MP4	H.264 / H.265	Lossy (block-based)	Most widely supported; efficient GPU decoding
AVI	DivX / XviD / Uncompressed	Can be Lossless or Lossy	Very large files; rarely used in ML pipelines due to size
MOV	ProRes, H.264	Lossy / Intra-frame	Used in Apple workflows; sometimes harder to integrate with Python libs
MKV	H.264, VP9, AV1	Lossy	Flexible container (subs, multiple audio); needs extra handling in ML pipelines





Video Resizing

- Standardization ensures every video has the same resolution (e.g., 224×224 for CNNs).
- Upscaling enlarges low-resolution frames (may cause blur).
- Downscaling reduces size for efficiency and avoids unnecessary high-res detail.
- Example: Resize all videos in dataset to 224×224 before feeding to model.



Color Space Conversion

- Videos use different color spaces: RGB, grayscale, YUV, HSV.
- Choice depends on task:
 - RGB → general vision tasks
 - Grayscale → reduces dimensionality
 - HSV → color-based object detection (e.g., skin, traffic signs)
- Example: Convert RGB video to grayscale for face recognition.



Frame Rate Adjustment

- Videos recorded at different FPS (24, 30, 60, 120).
- Standardizing FPS prevents mismatch when batching sequences.
- Downsampling: drop frames to lower FPS → faster training.
- Interpolation: generate missing frames → smooth playback or higher FPS.
- Example: Convert 60 FPS video to 30 FPS to reduce redundancy.



Frame-Level Processing

Frame Extraction

- A video = sequence of frames (images). Extracting frames makes video processing compatible with image-based pipelines.
- Typical tools: OpenCV (cv2.VideoCapture), FFmpeg.
- Extraction can be full (every frame) or partial (every nth frame).
- Example: 10s video @30 FPS → 300 frames.

Frame Sampling

- Frame sampling reduces redundancy and controls sequence length for DL models.
- Strategies:
 - Uniform sampling: evenly spaced frames.
 - Random sampling: introduces variety (good for augmentation).
 - Keyframe sampling: based on shot boundaries or motion intensity.
- Example: In sports video, uniform sampling may miss sudden key actions → keyframe sampling works better.



Temporal Processing

- Videos = not just frames, but temporal evolution of pixels/objects.
- Shot boundary detection: abrupt (cut) or gradual (fade, dissolve) transitions.
- Scene segmentation: groups shots into semantic events.
- Optical flow estimation: computes pixel motion vectors between frames.
- Tools: OpenCV (Farnebäck, Lucas-Kanade), DeepFlow, RAFT (DL-based).



Temporal Processing Techniques

Technique	Method	Use Case
Shot detection	Histogram difference, edge change ratio	Video summarization
Scene segmentation	Clustering shots, semantic labels	Video search, summarization
Optical flow	Lucas-Kanade, RAFT	Action recognition, motion tracking



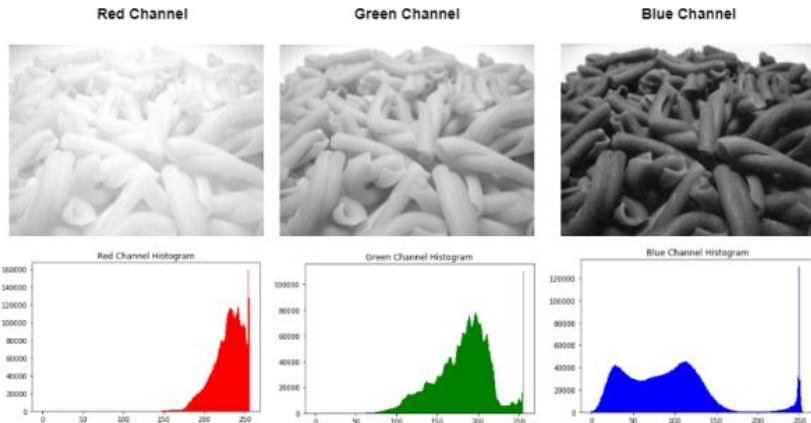
Data Augmentation in Videos

- Video augmentation applies transformations to increase dataset diversity while preserving temporal consistency.
- Augmentations are done either frame-by-frame (image-level ops applied consistently across all frames) or clip-level (temporal operations like skipping, reversing).
- Implemented using tools like OpenCV, FFmpeg, Torchvision, TensorFlow, NVIDIA DALI for efficient pipelines.
- Typically applied on-the-fly during training so each epoch sees new variations without storing huge augmented datasets.

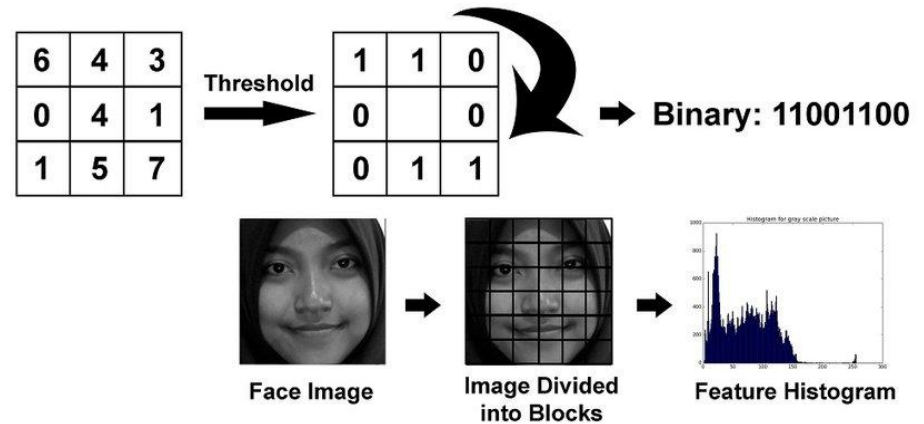


Video Feature Extraction

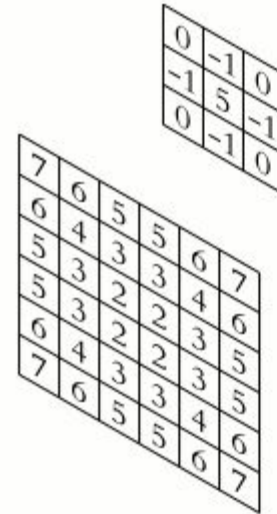
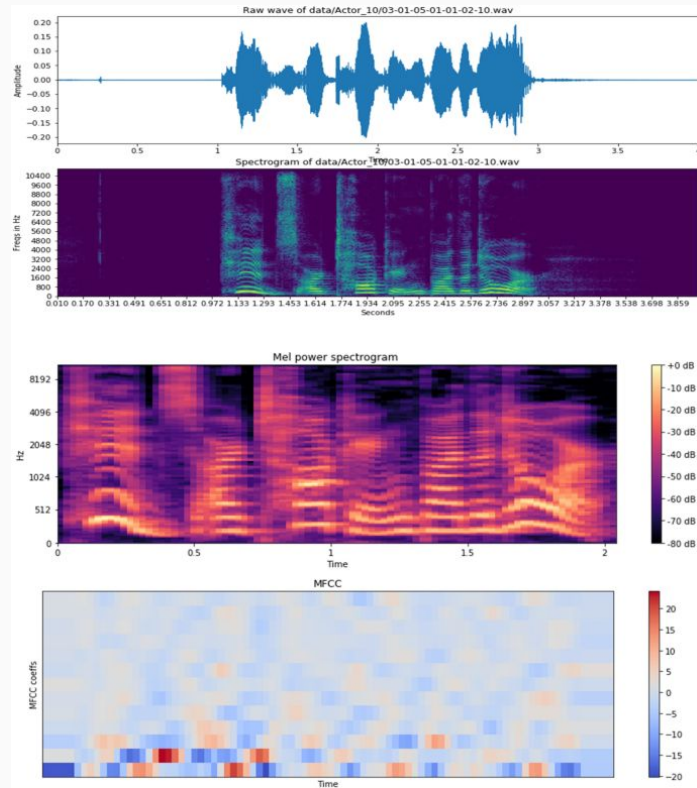
- Raw frames are too high-dimensional (e.g., a 10s video @ 30 FPS, 224×224 RGB → >450M values). Feature extraction reduces this into compact, discriminative representations.
- Features can be handcrafted (classical vision) or learned (deep networks).
- For deep learning pipelines, we usually pre-extract features with CNNs/3D CNNs and feed them into temporal models (RNNs, Transformers).
- Example: For a 10s clip, instead of 300 raw frames, store a sequence of 300 feature vectors (e.g., 2048-D each).



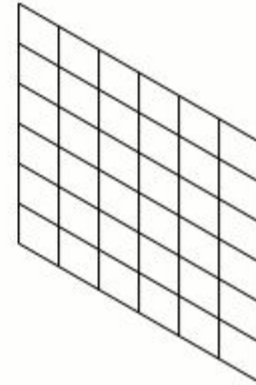
Color Moments: Statistical measures (mean, variance, skewness) of color distributions that capture the **color's central tendency**



LBPH - Local Binary Pattern Histogram



output





Pipeline for Video Feature Extraction

Preprocessing → resize, normalize frames/clips.

Frame/Clip Feature Extraction

- **Frame-level:** Pass each frame through 2D CNN (e.g., ResNet).
- **Clip-level:** Pass short clips (16/32 frames) through 3D CNN (e.g., I3D, C3D).

Feature Pooling → average/max pool across frames/clips to compress.

Storage / Integration → save as feature vectors (e.g., NumPy arrays) or feed directly into RNN/Transformer.



Introduction to DL for Video

- Unlike images, videos add a temporal dimension (T): data shape $\approx$ Batch $\times$ Channels $\times$ Time $\times$ Height $\times$ Width.
- Goal: learn spatial features (objects, textures) + temporal dynamics (motion, events).
- Challenge: large memory/computation (e.g., a 10s 30FPS clip can exceed GPU limits).
- Solution: sample fixed-length clips (e.g., 16/32 frames) for training.



Pipeline for Video DL Input

- Load video → sample clip (16–64 frames).
-
- Preprocess: resize, normalize, augment.
-
- Pack into tensor of shape (B, C, T, H, W).
-
- Feed into model (CNN+RNN, 3D CNN, Transformer).

CNN + RNN Architecture

- **CNN** (2D) extracts per-frame spatial features (e.g., ResNet on each frame).
- **RNN (LSTM/GRU)** models sequence over time, learning dependencies across frames.
- Works well for medium-length videos and tasks like captioning or gesture recognition.
- Limitation: sequential nature → slow training, vanishing gradients for long videos.



Pipeline (CNN+RNN)

1. Frames → 2D CNN → feature vector per frame.
2. Sequence of vectors → RNN/LSTM → temporal encoding.
3. Final hidden state → classifier or decoder.



3D CNNs

D convolutions apply filters over spatial + temporal dimensions simultaneously.

Each kernel spans (time × height × width) → captures motion + appearance.

Examples: C3D, I3D, R(2+1)D → benchmarks for video classification.

Limitation: GPU/memory heavy, requires large labeled datasets.



Recent Advancements

Transformers (TimeSformer, ViViT): handle long-range dependencies with self-attention.

Self-Supervised Learning (SSL): learn features without labels (predict next frame, contrastive tasks).

Multimodal Models: combine video with audio/text (e.g., VideoBERT, CLIP for video).

Generative Models: diffusion and GANs for video synthesis (e.g., text → video).

Question?